

VELARIS — Análisis de Rendimiento y Optimización

Velaris Dev Team

Versión 1.0 — 2025

Introducción

Este documento describe las decisiones de optimización implementadas en VELARIS, incluyendo los índices creados, el diseño de triggers, las funciones de seguridad y las estrategias adoptadas para garantizar un rendimiento adecuado a medida que el sistema escala.

1. Índices Implementados

Los índices son la principal herramienta de optimización en bases de datos relacionales. Reducen el tiempo de búsqueda de $O(n)$ a $O(\log n)$ al crear estructuras de árbol B que permiten localizar registros sin escanear la tabla completa.

1.1 Índices en (products)

```
CREATE INDEX idx_products_category ON velaris.products  
(category_id);  
  
CREATE INDEX idx_products_brand ON velaris.products (brand);  
  
CREATE INDEX idx_products_active ON velaris.products (active);
```

Justificación:

idx_products_category — las queries de catálogo siempre filtran por categoría. Sin índice, cada consulta haría un full scan de la tabla.

idx_products_brand — las búsquedas por marca son frecuentes en el punto de venta.

idx_products_active — casi todas las queries incluyen `WHERE active = TRUE`. Este índice evita escanear productos inactivos.

1.2 Índices en (purchase_orders)

```
CREATE INDEX idx_orders_supplier ON velaris.purchase_orders  
(supplier_id);  
CREATE INDEX idx_orders_status ON velaris.purchase_orders  
(status);
```

Justificación:

idx_orders_supplier — los reportes de compras filtran frecuentemente por proveedor.

idx_orders_status — filtrar órdenes por estado (pending, received) es una operación común en la gestión de compras.

1.3 Índices en (sales)

```
CREATE INDEX idx_sales_customer ON velaris.sales (customer_id);  
CREATE INDEX idx_sales_date ON velaris.sales (sale_date);  
CREATE INDEX idx_sales_employee ON velaris.sales (employee_id);
```

Justificación:

idx_sales_custome — el historial de compras por cliente es una consulta frecuente.

idx_sales_date — los reportes de ventas siempre incluyen rangos de fecha.

idx_sales_employee — ****crítico para el RLS****: la política del rol (seller) ejecuta **WHERE employee_id = fn_current_employee_id()** en cada consulta. Sin este índice, cada acceso del seller haría un full scan de la tabla (sales)

1.4 Índices Parciales en (inventory_movements)

```
CREATE INDEX idx_movements_sale  
ON velaris.inventory_movements (sale_id)  
WHERE sale_id IS NOT NULL;
```

```
CREATE INDEX idx_movements_purchase_order
ON velaris.inventory_movements (purchase_order_id)
WHERE purchase_order_id IS NOT NULL;
```

Justificación:

Estos son **índices parciales** — solo indexan las filas donde la columna no es NULL. Esto es más eficiente que un índice normal porque:

- Los movimientos de tipo (adjustment) y (return) no tienen (sale_id) ni (purchase_order_id)
- Un índice parcial es más pequeño y rápido que uno completo
- Las búsquedas de trazabilidad (¿qué movimientos generó esta venta?) son exactamente el caso de uso

1.5 Índices en (audit_log)

```
CREATE INDEX idx_audit_table ON velaris.audit_log
(affected_table);
CREATE INDEX idx_audit_date ON velaris.audit_log (recorded_at);
```

Justificación:

La tabla (**audit_log**) crecerá indefinidamente con el tiempo — es la tabla que más registros acumulará. Sin índices, las consultas de auditoría serían cada vez más lentas. (**idx_audit_table**) permite filtrar por tabla afectada y (**idx_audit_date**) permite filtrar por rangos de fecha eficientemente.

2. Diseño de Triggers para Rendimiento

2.1 Separación de responsabilidades

El trigger de ventas fue diseñado con una separación clara de responsabilidades para evitar operaciones duplicadas:

Trigger	Tabla	Momento	Responsabilidad
trg_validate_sale_stock	sales_details	AFTER INSERT	Solo valida stock disponible
trg_update_stock	inventory_movements	AFTER INSERT	Solo actualiza (current_stock)

Problema que resuelve:

una implementación naive podría descontar el stock tanto al insertar el detalle de venta como al insertar el movimiento, resultando en un doble descuento. La separación garantiza que el stock se modifica exactamente una vez por operación.

2.2 Funciones de trigger con SECURITY DEFINER

Todas las funciones de trigger usan (SECURITY DEFINER):

```
CREATE OR REPLACE FUNCTION velaris.fn_update_stock_trigger()
RETURNS TRIGGER AS $$
...
$$ LANGUAGE plpgsql SECURITY DEFINER;
```

Justificación:

(SECURITY DEFINER) permite que el trigger se ejecute con los permisos del creador de la función (superusuario), no del usuario que disparó la operación. Esto es necesario porque el RLS restringe el acceso de los usuarios normales a ciertas tablas, pero los triggers deben poder actualizar (products.current_stock) independientemente del rol del usuario.

2.3 Trigger de (updated_at) genérico

En lugar de crear una función diferente para cada tabla, se reutiliza una sola función:

```
CREATE OR REPLACE FUNCTION velaris.fn_updated_at_trigger()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

Justificación:

reduce la cantidad de código duplicado y facilita el mantenimiento. Si se necesita cambiar el comportamiento del (updated_at), se modifica en un solo lugar.

3. Row Level Security (RLS) Optimizado

3.1 Funciones helper estables

Las políticas RLS no ejecutan subqueries directamente, sino que delegan a funciones marcadas como (STABLE):

```
CREATE OR REPLACE FUNCTION velaris.fn_current_user_role()
RETURNS velaris.user_role
LANGUAGE sql STABLE SECURITY DEFINER AS $$
    SELECT role FROM velaris.system_users
    WHERE auth_user_id = auth.uid() LIMIT 1;
$$;
```

Justificación:

marcar la función como (STABLE) le indica a PostgreSQL que el resultado no cambia dentro de una misma transacción. Esto permite que el planificador de consultas **cachee el resultado (guardé temporalmente el resultado de una operación para no tener que calcularlo o consultarlo otra vez.)**

de (**fn_current_user_role()**) durante toda la transacción, evitando ejecutar el subquery contra (**system_users**) una vez por cada fila evaluada.

3.2 Política del seller con índice de soporte

La política del seller en `sales` es:

```
USING (
    velaris.fn_current_user_role() = 'seller'
    AND employee_id = velaris.fn_current_employee_id()
)
```

Esta política se beneficia directamente del índice (**idx_sales_employee**). Sin él, PostgreSQL haría un sequential scan de toda la tabla (**sales**) para cada consulta del seller. Con el índice, localiza las filas relevantes en `**O(log n)**`.

4. Procedimientos Almacenados para Operaciones Complejas

4.1 Transaccionalidad implícita

Los stored procedures (**sp_register_purchase**) y (**sp_register_sale**) agrupan múltiples operaciones en una sola llamada:

```
sp_register_sale()
├— Validar customer
├— Validar employee
├— Validar warehouse
├— INSERT INTO sales
└— Para cada producto:
    ├── INSERT INTO sales_details → dispara trg_validate_sale_stock
    └── INSERT INTO inventory_movements → dispara trg_update_stock
```

Justificación:

en lugar de hacer múltiples round-trips entre la aplicación y la base de datos, una sola llamada al SP ejecuta toda la operación. Esto reduce la latencia de red y garantiza que la operación sea atómica — si falla en cualquier punto, no quedan datos inconsistentes.

4.2 Validaciones tempranas

Las validaciones de existencia se hacen al inicio del SP antes de cualquier INSERT:

```
IF NOT EXISTS (SELECT 1 FROM velaris.warehouses WHERE
warehouse_id = p_warehouse_id AND active = TRUE) THEN
    RAISE EXCEPTION 'Warehouse ID % does not exist or is inactive',
p_warehouse_id;
END IF;
```

Justificación:

fallar rápido evita ejecutar trabajo innecesario. Si el warehouse no existe, no tiene sentido crear la orden de compra, los detalles y los movimientos para luego hacer rollback.

5. Diseño de Trazabilidad sin Overhead

Las columnas (**sale_id**) y (**purchase_order_id**) en (**inventory_movements**) son opcionales (nullable):

<pre>sale_id INT REFERENCES velaris.sales(sale_id), purchase_order_id INT REFERENCES velaris.purchase_orders(order_id)</pre>

Justificación:

en lugar de crear una tabla intermedia de trazabilidad (que requeriría JOINS adicionales en cada consulta), se agregaron FKs directamente a la tabla de movimientos. Los índices parciales garantizan que estas columnas no degraden el rendimiento en movimientos que no tienen origen en ventas o compras.

6. Normalización hasta 4FN

El schema cumple con la Cuarta Forma Normal (4FN):

1FN: cada celda contiene un valor atómico — no hay columnas con listas o grupos repetidos

2FN: cada columna no clave depende completamente de la clave primaria — (**sales_details**) separa los productos de la venta

3FN: no hay dependencias transitivas — los datos del cliente no están en (**sales**), sino en (**customers**)

4FN: no hay dependencias multivaluadas independientes — cada entidad tiene su propia tabla

Impacto en rendimiento:

la normalización reduce la redundancia de datos, lo que disminuye el tamaño de las tablas y mejora el rendimiento de las operaciones de escritura. Las lecturas se optimizan con los índices descritos anteriormente.

7. Resumen de Optimizaciones

Optimización	Impacto	Tipo
13 índices estratégicos	Reduce tiempo de búsqueda de $O(n)$ a $O(\log n)$	Lectura
Índices parciales en movements	Índices más pequeños y rápidos	Lectura
Funciones RLS STABLE	Evita ejecutar subquery por cada fila	RLS
SECURITY DEFINER en triggers	Evita fallos de permisos en operaciones automáticas	Seguridad
SPs con validaciones tempranas	Fail-fast evita trabajo innecesario	Escritura
Una sola función de updated_at	Reduce código duplicado y mantenimiento	Mantenimiento
Separación trigger validación/actualización	Elimina doble descuento de stock	Integridad
Normalización 4FN	Reduce redundancia y tamaño de tablas	General